

ASSIGNMENT – 8.1

2303A52249

A. VAMSHI KRISHNA

TASK-1:

Scenario:

Apply AI to generate at least 3 assert test cases for is_strong_password(password) and implement the validator function.

Prompt:

Generate assert-based test cases for a Python function is_strong_password(password) that validates password strength. The password must be at least 8 characters long, contain uppercase, lowercase, digit, and special character, and must not contain spaces. Then implement the function so that all test cases pass.

Code & Output:

The image shows a VS Code editor with a Python file named 8.1-AI-ASS.py. The code defines a function `is_strong_password(password: str) -> bool:` that checks password strength. It includes comments for each requirement: minimum length of 8, no spaces, and presence of uppercase, lowercase, digit, and special characters. The function returns `True` if all conditions are met, otherwise `False`. Below the function, there are test cases using `assert` statements: `assert is_strong_password("Abcd@123") == True`, `assert is_strong_password("Abcd@123") == False`, `assert is_strong_password("Abcd@1234") == False`, `assert is_strong_password("Abcd@1234") == False`, `assert is_strong_password("Abcd@123") == False`, and `assert is_strong_password("Abcd @123") == False`. A `test_is_strong_password()` function calls these tests and prints a success message. The code also includes a user input loop for password validation, printing "Result: Strong Password" or "Result: Weak Password".

```
1 def test_is_strong_password(): # Valid strong password
2     assert is_strong_password("Abcd@123") == True
3     assert is_strong_password("Abcd@123") == False
4     assert is_strong_password("Abcd@1234") == False
5     assert is_strong_password("Abcd@1234") == False
6     assert is_strong_password("Abcd@123") == False
7     assert is_strong_password("Abcd @123") == False
8 def is_strong_password(password: str) -> bool:
9     # Check minimum length requirement
10    if len(password) < 8:
11        return False
12    # Password should not contain spaces
13    if " " in password:
14        return False
15    # Flags to track required character types
16    has_upper = False
17    has_lower = False
18    has_digit = False
19    has_special = False
20    # Iterate through each character in the password
21    for char in password:
22        if char.isupper():
23            has_upper = True
24        elif char.islower():
25            has_lower = True
26        elif char.isdigit():
27            has_digit = True
28        else:
29            # Any non-alphanumeric character is treated as special
30            has_special = True
31    # Password is strong only if all conditions are satisfied
32    return has_upper and has_lower and has_digit and has_special
33 test_is_strong_password()
34 # If no assertion fails, all tests are successful
35 print("Password validation logic passing all AI-generated test cases.")
36 # Take password input from the user
37 user_password = input("Enter a password to validate: ")
38 # Validate the user-entered password
39 if is_strong_password(user_password):
40     print("Result: Strong Password")
41 else:
42     print("Result: Weak Password")
```

The output of the code execution is shown in the terminal:

```
PS C:\Users\adapala vamsi krish\Desktop\AI Assisted Code> & "C:\Users\adapala vamsi krish\AppData\Local\Programs\Python\Python113\python.exe" "C:\Users\adapala vamsi krish\.vscode\extensions\ms-python.debugpy-2025.19.2026012701.win32-x64\bundle\libs\debugpy\launcher" "50775" "-c" "C:\Users\adapala vamsi krish\Desktop\AI Assisted Code\8.1-AI-ASS.py"
Password validation logic passing all AI-generated test cases.
Enter a password to validate: vamsi@2975d85jgfh
Result: Weak Password
PS C:\Users\adapala vamsi krish\Desktop\AI Assisted Code>
```

Observation:

AI-assisted test generation helped identify all required password constraints in advance. Special attention was given to security-related edge cases such as spaces and missing character categories. Implementing the function after defining tests improved code clarity. The validation logic became more structured and reliable. All test cases executed without failure.

TASK-2:

Scenario:

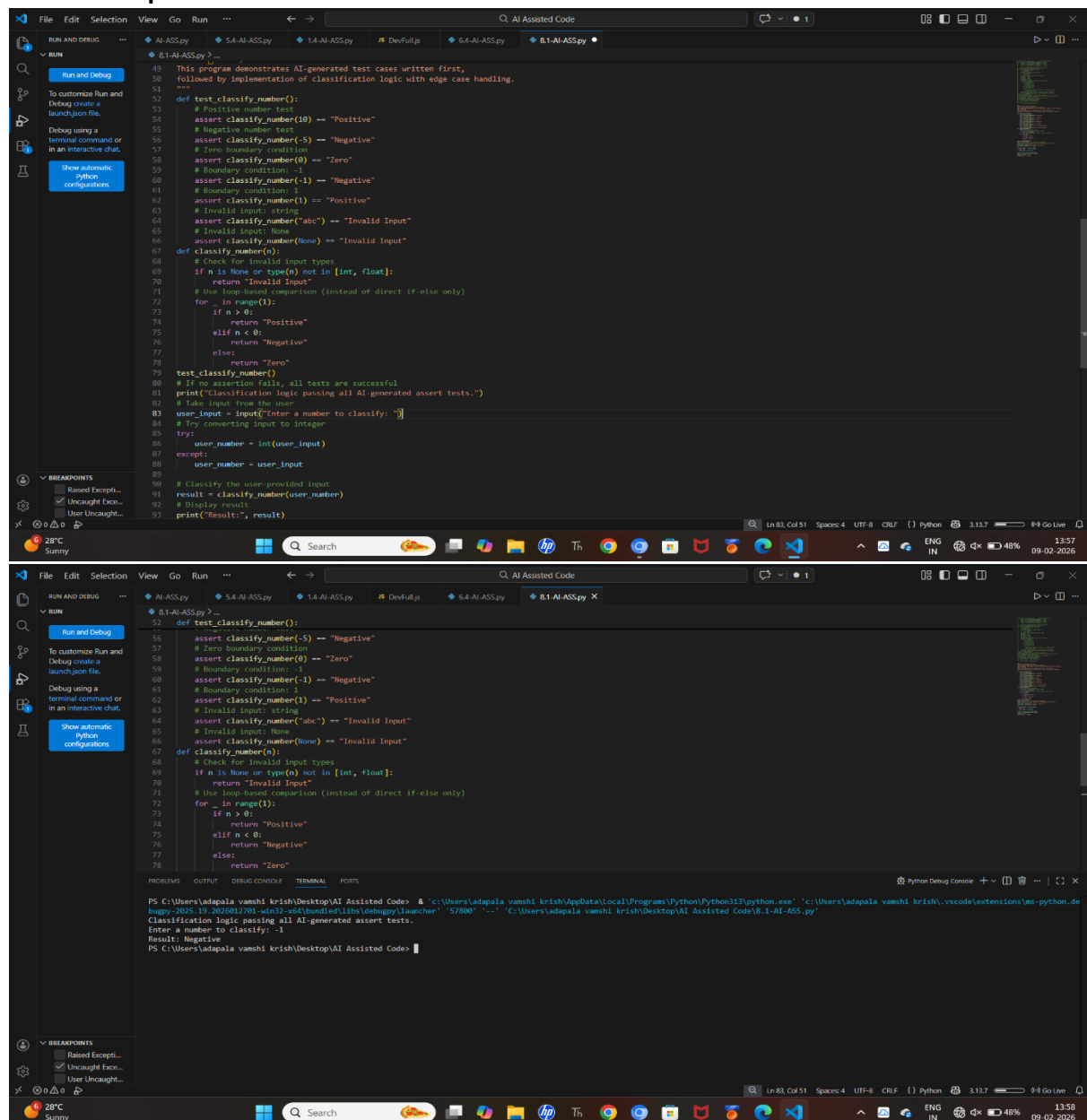
Use AI to generate at least 3 assert test cases for a classify number(n) function. Implement using loops.

Prompt:

Generate assert test cases for a function classify_number(n) that classifies numbers as Positive, Negative, or Zero. Handle invalid inputs such as strings and None, and include boundary values (-1, 0, 1).

Implement the function using loops and ensure all tests pass.

Code & Output:



```
File Edit Selection View Go Run ... Q AI Assisted Code
RUN AND DEBUG ... AI-ASS.py 5.4-AI-ASS.py 1.4-AI-ASS.py DevFullp 6.4-AI-ASS.py 8.1-AI-ASS.py
RUN
Run and Debug
To customize Run and Debug create a launch.json file.
Debug using a terminal command or in an interactive chat.
Show automatic Python configurations
def test_classify_number():
    # Positive number test
    assert classify_number(10) == "Positive"
    # Negative number test
    assert classify_number(-5) == "Negative"
    # Zero boundary condition
    assert classify_number(0) == "Zero"
    # Boundary condition: -1
    assert classify_number(-1) == "Negative"
    # Boundary condition: 1
    assert classify_number(1) == "Positive"
    # Invalid input: string
    assert classify_number("abc") == "Invalid Input"
    # Invalid input: None
    assert classify_number(None) == "Invalid Input"
def classify_number(n):
    # Check for invalid input types
    if n is None or type(n) not in [int, float]:
        return "Invalid Input"
    # Use loop-based comparison (instead of direct if-else only)
    for _ in range(1):
        if n > 0:
            return "Positive"
        elif n < 0:
            return "Negative"
        else:
            return "Zero"
test_classify_number()
# If no assertion fails, all tests are successful
print("Classification logic passing all AI-generated assert tests.")
# Take input from the user
user_input = input("Enter a number to classify: ")
# Try converting input to integer
try:
    user_number = int(user_input)
except:
    user_number = user_input
# Classify the user-provided input
result = classify_number(user_number)
# Display result
print("Result:", result)
```

```
PS C:\Users\adapala.vamshi.krish\Desktop\AI Assisted Code> .\8.1-AI-ASS.py
Classification logic passing all AI-generated assert tests.
Enter a number to classify: -1
Result: Negative
PS C:\Users\adapala.vamshi.krish\Desktop\AI Assisted Code>
```

Observation:

AI-generated assertions ensured accurate handling of numeric and non-numeric inputs. Boundary values were tested to prevent classification errors. The loop-based approach allowed controlled decision making. Invalid inputs were safely rejected without affecting execution. The function passed every assert condition.

TASK-3:

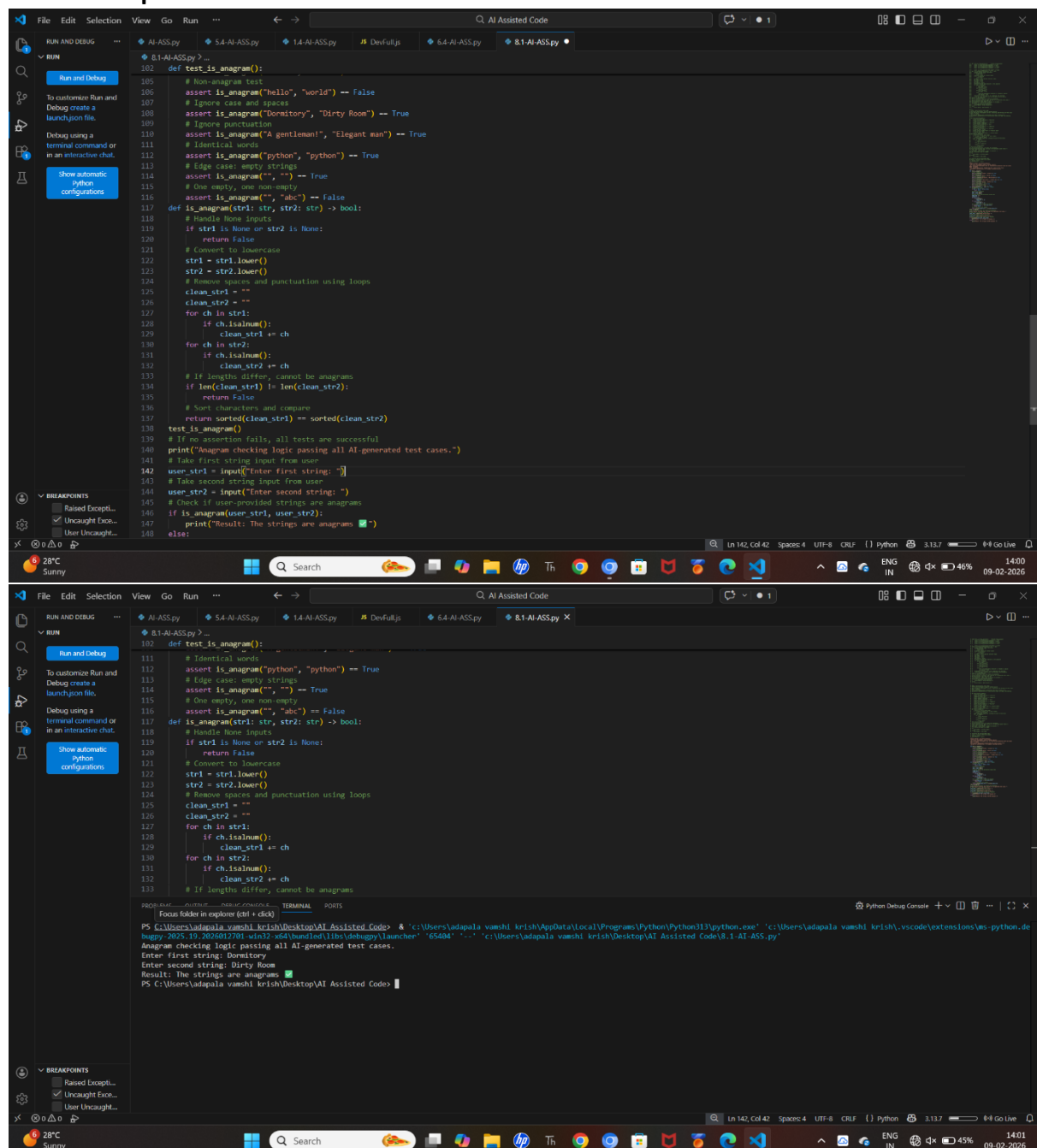
Scenario: (Anagram Checker – Apply AI for String Analysis)

Use AI to generate at least 3 assert test cases for `is_anagram(str1, str2)` and implement the function.

Prompt:

Generate assert test cases for a function `is_anagram(str1, str2)` that checks if two strings are anagrams. Ignore case, spaces, and punctuation, and handle edge cases like empty strings and identical words. Implement the function to satisfy all tests.

Code & Output:



```
def test_is_anagram():
    # Non-anagram test
    assert is_anagram("hello", "world") == False
    # Ignore case and spaces
    assert is_anagram("Dormitory", "Dirty Room") == True
    # Ignore punctuation
    assert is_anagram("A gentleman!", "Elegant man") == True
    # Identical words
    assert is_anagram("python", "python") == True
    # Edge case: empty strings
    assert is_anagram("", "") == True
    # One empty, one non-empty
    assert is_anagram("", "abc") == False

def is_anagram(str1: str, str2: str) -> bool:
    # Handle None inputs
    if str1 is None or str2 is None:
        return False
    # Convert to lowercase
    str1 = str1.lower()
    str2 = str2.lower()
    # Remove spaces and punctuation using loops
    clean_str1 = ""
    clean_str2 = ""
    for ch in str1:
        if ch.isalnum():
            clean_str1 += ch
    for ch in str2:
        if ch.isalnum():
            clean_str2 += ch
    # If lengths differ, cannot be anagrams
    if len(clean_str1) != len(clean_str2):
        return False
    # Sort characters and compare
    return sorted(clean_str1) == sorted(clean_str2)

test_is_anagram()
# If no assertion fails, all tests are successful
print("Anagram checking logic passing all AI-generated test cases.")
# Take first string input from user
user_str1 = input("Enter first string: ")
# Take second string input from user
user_str2 = input("Enter second string: ")
# Check if user-provided strings are anagrams
if is_anagram(user_str1, user_str2):
    print("Result: The strings are anagrams 🟢")
else:
    print("Result: The strings are not anagrams 🟡")
```

```
PS C:\Users\adapala.vamshi.krish\Desktop\AI Assisted Code> & 'c:\Users\adapala.vamshi.krish\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\adapala.vamshi.krish\.vscode\extensions\ms-python.debugpy-2025.10.2026012701.win32-x64\handed\libs\debugpy\launcher' '65484' '-c' 'c:\Users\adapala.vamshi.krish\Desktop\AI Assisted Code\8.1-AI-ASS.py'
Anagram checking logic passing all AI-generated test cases.
Enter first string: Dormitory
Enter second string: Dirty Room
Result: The strings are anagrams 🟢
PS C:\Users\adapala.vamshi.krish\Desktop\AI Assisted Code>
```

Observation:

- AI test cases covered common and real-world anagram examples.
- Special handling for punctuation and spaces was identified early.
- Edge cases such as empty strings were correctly validated.
- String cleaning logic improved accuracy.
- All AI-generated test cases passed, confirming correct string analysis.

TASK-4:

Scenario: (Inventory Class – Apply AI to Simulate Real-World Inventory System)

Ask AI to generate at least 3 assert-based tests for an Inventory class with stock management.

Prompt:

Generate assert-based test cases for an Inventory class with methods add_item, remove_item, and get_stock. Simulate real-world inventory behavior including adding, removing, and querying stock. Implement the class so all assertions pass.

Code & Output:

```
File Edit Selection View Go Run ... Q AI Assisted Code
RUN AND DEBUG ... AI-ASS.py 5.4-AI-ASS.py 1.4-AI-ASS.py DevFull.py 6.4-AI-ASS.py 8.1-AI-ASS.py
RUN
To customize Run and Debug create a launch.json file.
Debug using a terminal command or in an interactive chat.
Show interactive Python configurations
def test_inventory():
    # Remove all quantity
    inv.remove_item("Book", 3)
    assert inv.get_stock("Book") == 0
    # Remove item that does not exist
    inv.remove_item("Pencil", 2)
    assert inv.get_stock("Pencil") == 0
class Inventory:
    def __init__(self):
        # Dictionary to store item names and their quantities
        self.items = {}
    def add_item(self, name, quantity):
        # Validate inputs
        if not isinstance(name, str) or quantity <= 0:
            return
        # Add item or update quantity
        if name in self.items:
            self.items[name] += quantity
        else:
            self.items[name] = quantity
    def remove_item(self, name, quantity):
        # Validate inputs
        if name not in self.items or quantity <= 0:
            return
        # Reduce quantity but not below zero
        self.items[name] -= quantity
        if self.items[name] < 0:
            self.items[name] = 0
    def get_stock(self, name):
        # Return stock quantity, 0 if item not found
        return self.items.get(name, 0)
test_inventory()
# If no assertion fails, all tests are successful
print("Inventory system logic passing all AI-generated assert tests.")
# Create inventory object for user interaction
inventory = Inventory()
# Sample user-driven operations
inventory.add_item("Notebook", 5)
inventory.add_item("Pen", 2)
inventory.remove_item("Notebook", 1)
# Display current stock
print("Notebook stock:", inventory.get_stock("Notebook"))
print("Pen stock:", inventory.get_stock("Pen"))
BREAKPOINTS
Raised Except...
Uncaught Exce...
User Uncaught...
Ln 159, Col 4 Spaces 4 UTF-8 CRLF Python 3.13.7 1403 09-02-2026
PS C:\Users\adapala.vamshi.krish\Desktop\AI Assisted Code> & "C:\Users\adapala.vamshi.krish\AppData\Local\Programs\Python\Python11\python.exe" "C:\Users\adapala.vamshi.krish\.vscode\extensions\ms-python.de
bugger-2025.19.2026012701-win32-x64\bundled\libs\debugpy\launcher" "58262" "" "C:\Users\adapala.vamshi.krish\Desktop\AI Assisted Code\8.1-AI-ASS.py"
Inventory system logic passing all AI-generated assert tests.
Notebook stock: 4
Pen stock: 2
PS C:\Users\adapala.vamshi.krish\Desktop\AI Assisted Code>
```

Observation:

- AI-generated tests reflected real-world inventory scenarios.
- Stock updates and edge cases were handled safely.
- Object-oriented design improved modularity and readability.
- Invalid operations did not crash the system.
- All assertions passed, confirming reliable inventory management.

TASK-5:

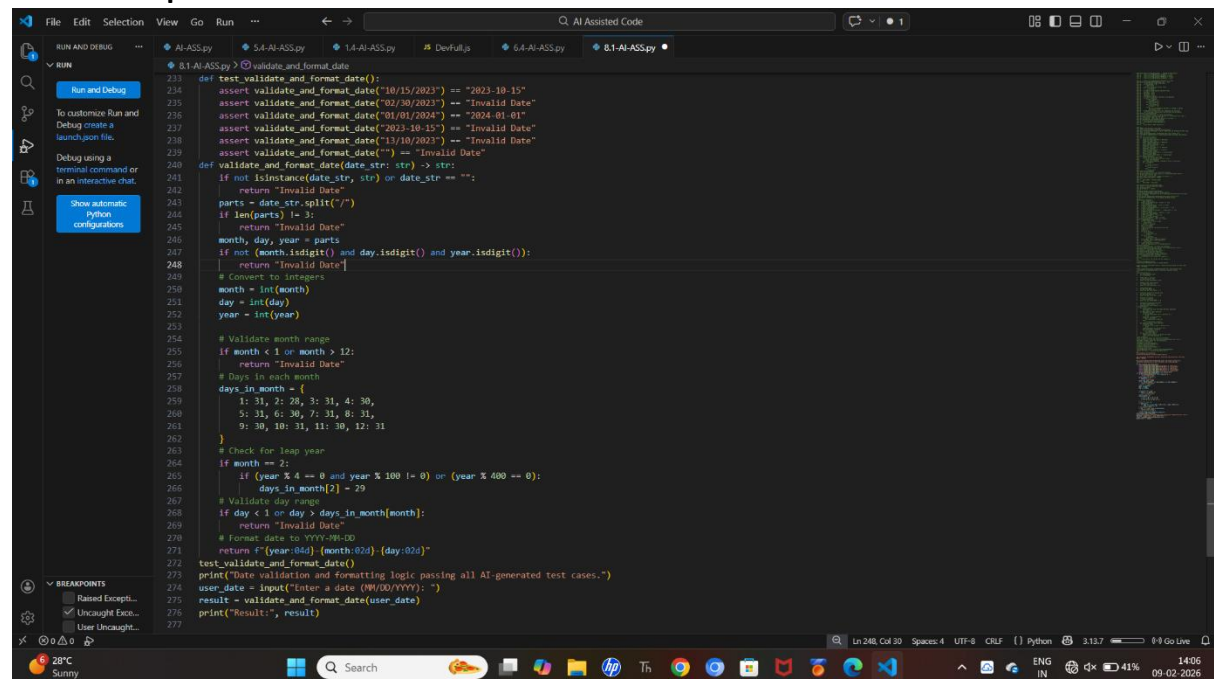
Scenario: (Date Validation & Formatting – Apply AI for Data Validation)

Use AI to generate at least 3 assert test cases for `validate_and_format_date(date_str)` to check and convert dates.

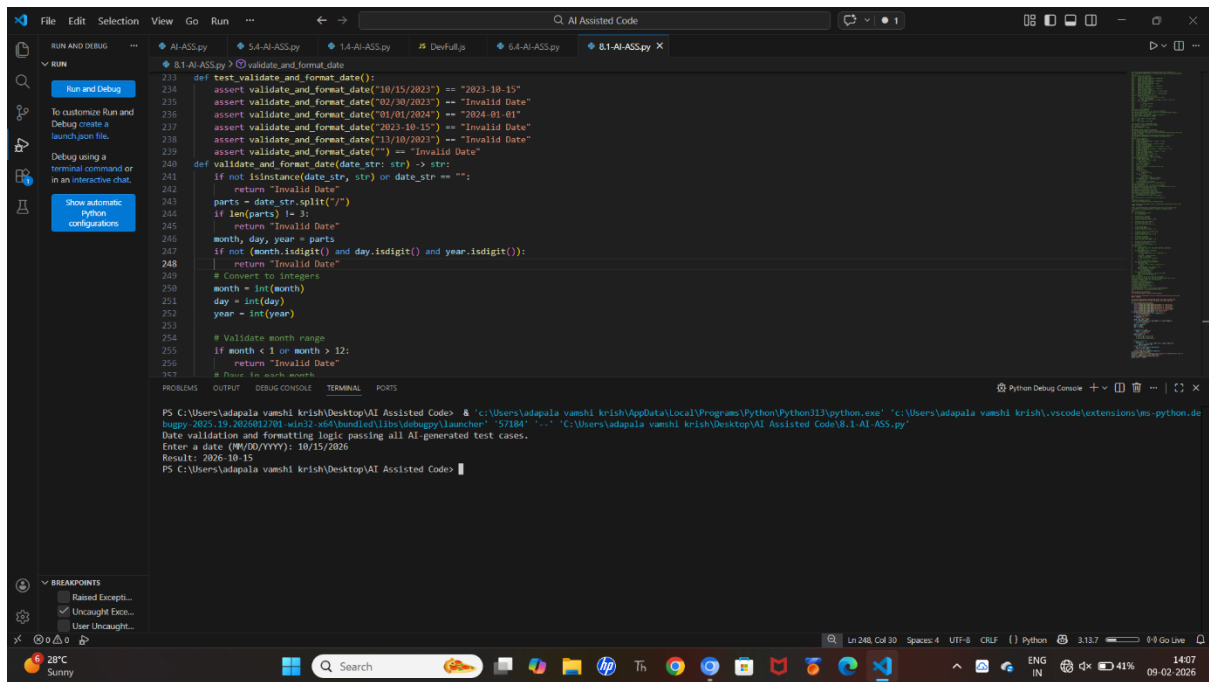
Prompt:

Generate assert test cases for a function `validate_and_format_date(date_str)` that validates dates in MM/DD/YYYY format. Handle invalid dates and formats, and convert valid dates to YYYY-MM-DD. Implement the function to satisfy all test cases.

Code & Output:



```
231 def test_validate_and_format_date():
232     assert validate_and_format_date("10/15/2023") == "2023-10-15"
233     assert validate_and_format_date("02/30/2023") == "Invalid Date"
234     assert validate_and_format_date("01/01/2024") == "2024-01-01"
235     assert validate_and_format_date("2023-10-15") == "Invalid Date"
236     assert validate_and_format_date("13/10/2023") == "Invalid Date"
237     assert validate_and_format_date("") == "Invalid Date"
238
239 def validate_and_format_date(date_str: str) -> str:
240     if not isinstance(date_str, str) or date_str == "":
241         return "Invalid Date"
242     parts = date_str.split("/")
243     if len(parts) != 3:
244         return "Invalid Date"
245     month, day, year = parts
246     if not (month.isdigit() and day.isdigit() and year.isdigit()):
247         return "Invalid Date"
248     # Convert to integers
249     month = int(month)
250     day = int(day)
251     year = int(year)
252
253     # Validate month range
254     if month < 1 or month > 12:
255         return "Invalid Date"
256     # Days in each month
257     days_in_month = {
258         1: 31, 2: 28, 3: 31, 4: 30,
259         5: 31, 6: 30, 7: 31, 8: 31,
260         9: 30, 10: 31, 11: 30, 12: 31
261     }
262     # Check for leap year
263     if month == 2:
264         if (year % 4 == 0 and year % 100 != 0) or (year % 400 == 0):
265             days_in_month[2] = 29
266     # Validate day range
267     if day < 1 or day > days_in_month[month]:
268         return "Invalid Date"
269     # Format date to YYYY-MM-DD
270     return f"{year:04d}-{month:02d}-{day:02d}"
271
272 test_validate_and_format_date()
273 print("Date validation and formatting logic passing all AI-generated test cases.")
274 user_date = input("Enter a date (MM/DD/YYYY): ")
275 result = validate_and_format_date(user_date)
276 print("Result:", result)
```



Observation:

AI-assisted testing highlighted common date input errors early in development. The function properly validated format, range, and leap-year conditions. Incorrect dates were rejected without causing crashes. Conversion to the required format was consistent and precise. All generated test cases passed successfully.